

C++

1. What is C++?

- It is a general purpose Intermediate level Object oriented programming language.)
- C base ^{C++}, and advanced level of C.)

2. Why to use C++?

- C++ Can be found in today's operating systems, Graphical User Interfaces, and embedded systems.)
- C++ is an object oriented programming language which gives a clean structure to programs and allows code to be reused, lowering development costs.)
- C++ is portable and can be used to develop applications that can be adapted to multiple platforms.)
- C++ is easy to learn.)
- As C++ is close to C and Java, it makes it easy for programmers to switch to C++ or Vice versa.)

3. C++ Uses:-

Uses:-

(OS)

(Editors)

(Commercial applications)

(Database)

(Translator)

(Protocols)

++C

4. C++ History:-

(C++ Programming language was developed in 1980 by Bjarne Stroustrup at Bell Laboratories of AT & T located in U.S.A.)

5. C VS C++ :-

C

- It follows the procedural style programming.
- data is less secured in C.

C++

- It is multi-paradigm. It supports both Procedural and object oriented.
- In C++, you can use modifiers for class members to make it inaccessible for outside users.

C

(It follows the top-down approach)

C++

(It follows the bottom-up approach.)

C

(It does not support function overloading.)

C++

(It supports function overloading.)

C

(It does not support reference variables.)

C++

(supports reference variables.)

C

(in C, scanf() and printf() are mainly used for input/output.)

C++

(C++ mainly uses stream cin and cout to perform input and output operations.)

C

(operator overloading is not possible in C.)

C++

(operator overloading ^(=, &) is possible in C++.)

C

(C programs are divided into procedures and modules.)

C++

→ Programs
(C++ Programmers are divided into functions and classes.)

C

(C does not provide the feature of namespace.)

C++

(C++ supports the features of namespace.)

C

(Exception handling is not easy in C. It has to perform using other functions.)

C++

(C++ provides exception handling using Try and Catch block.)

6. C++ features:-

① Simple:-

→ C++ is a simple language in the sense that it provides structured approach to break the problem into parts, rich set of library functions, data types etc.)

② Machine Independent or Portable:-

→ unlike assembly language, C programs can be executed in many machines with little bit or no change. But it is not platform-independent.)

③ Mid-level Programming Language:-

C++ is also used to do low level programming. It is used to develop system applications such as kernel, driver etc. It also supports the feature of high level language. That is why it is known as mid-level language.)

④ Structured Programming Language:-

C++ is a structured programming language in the sense that we can break the program into parts using functions. So, it is easy to understand and modify.)

⑤ Rich Library:-

C++ provides a lot of inbuilt functions that makes the development fast.)

⑥ Memory Management:-

It supports the features of dynamic memory allocation. In C++ language, we can free calling the free() function.)

7) Speed :-

The compilation and execution time of C++ language is fast.)

8) Pointer :-

C++ provides the feature of pointers. We can directly interact with the memory by using the pointers. We can use pointers for memory, structures, functions, array etc.)

9) Recursion :-

In C++, we can call the function within the function. It provides code reusability for every function.)

10) Extensible :-

C++ language is extensible because it can easily adopt new features.)

11) Object Oriented :-

C++ is object oriented programming language. OOPS makes development and maintenance easier where as in procedure - oriented programming language it is not easy to manage if code grows as project size grows.)

12) Compiler based :-

C++ is a compiler based programming language, it means without compilation no C++ program can be executed. First we need to compile our program using compiler and then we can execute our program.)

13) Case sensitive :-

Just like C, it is pretty clear that the C++ programming language treats the uppercase and lowercase characters in a different manner. For instance, the meaning of the keyword 'count' changes if we write it as '~~count~~' or '~~count~~'. Other programming languages like HTML and MySQL are not case sensitive.

7. C++ Variables:-

- A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.)
- It is a way to represent memory location through symbol so that it can be easily identified.)

(Type Variable - name;)

(Type Variable 1 - name, Variable 2 - name, Variable 3 - name;)

• `int x;`
`float y;`
`char z;` } Variable // variable declare

• `int x = 5, b = 10;`
`float f = 30.8;`
`char c = 'A';` // Variable initialization

Rules for defining Variables:-

- A Variable can have alphabets, digits and underscore.)
- A variable name can start with alphabet)

and underscore only. It can't start with digits.)

- No white space is allowed within variable name.)
- A variable name must not be any reserved word or keyword e.g. char, float etc.)

```
(#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int age;
```

```
    cout << "Enter your age:";
```

```
    cin >> age;
```

```
    cout << "your age is : " << age << endl;
```

```
}
```

Output:-

Enter your age : 20

Your age is : 20

8. C++ Input & Output:-

(C++ Basic Input / Output:-

- C++ I/O operation is using the stream concept. Stream is the sequence of bytes or flow of data. It makes the performance fast.)

(I/O Library Header Files:-

<iostream>

It is used to define the cout, cin and cerr objects, which correspond to standard output stream, standard input stream and standard error stream, respectively.)

<iomanip> → Manipulator

It is used to declare services useful for performing formatted I/O, such as Set Precision and Set w.)

<fstream>

It is used to declare services for user-controlled file processing.)

Standard Output Stream (cout):-

The cout is a predefined object of ostream Class. It is connected with the standard output device, which is usually a display screen. The cout is used in conjunction with the stream insertion operator (<<) to display the output on a console.)

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    char any[] = "welcome to programming line";
```

```
    cout << "value of any is : " << any << endl;
```

```
}
```

Output:-

Welcome to programming line.

Standard Input Stream (cin):-

The cin is a predefined object of istream Class. It is connected with the standard input device, which is usually a keyboard.

The cin is used in conjunction with stream extraction operator (>>) to read the input from a console.)

```
#include <iostream>
using namespace std;
int main() {
    int age;
    cout << "Enter your age:";
    cin >> age;
    cout << "Your age is: " << age << endl;
}
```

Output:-

Enter your age: 17

Your age is: 17

(Standard end line (endl):-

The endl is a predefined object of ostream class. It is used to insert a new line character and flushes the stream.)

```
#include <iostream>
using namespace std;
int main() {
```

```
    cout << "Programming";
```

```
    cout << "Line " << endl;
```

```
}
```

Output:-

Programming Line

9. C++ Operators Basics:-

(C++ Operators:-

An operator is simply a symbol that is used to perform operations.)

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operator
- Unary Operator
- Ternary or Conditional Operator
- Misc Operator)

• Binary Operation

+, -, *, /, % Arithmetic Operators

<, <=, >, >=, ==, != Relational Operators

&, ||, ! Logical Operators

&, |, <<, >>, ~, ^ Bitwise Operators

=, +=, -=, *=, /=, %= Assignment Operators

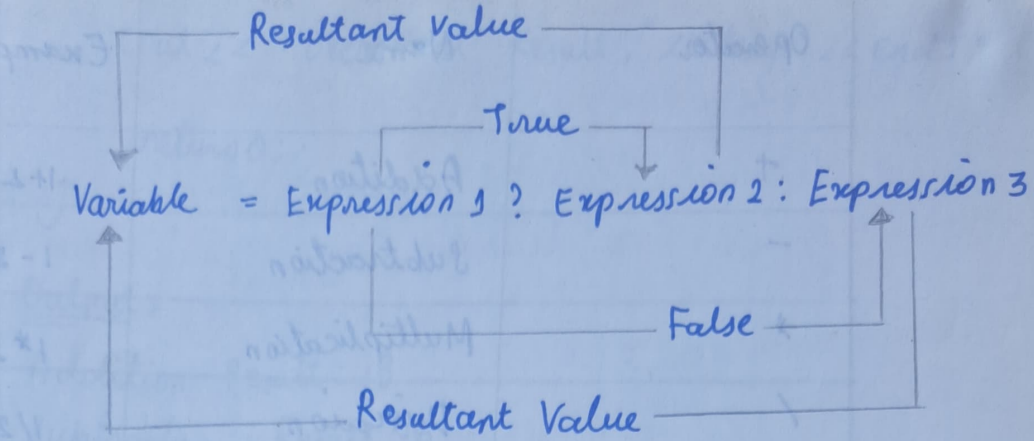
• Unary Operator $\rightarrow ++, --$

• Ternary Operator $\rightarrow ? :$

10. C++ Conditional Operators:-

Conditional or Ternary Operator ($? :$)

The conditional operator is kind of similar to the if-else statement as it does follow the same algorithm as of if-else statement but the conditional operator takes less space and helps to write the if-else statements in the shortest way possible.



```

#include <iostream>
using namespace std;
int main()
{
    int n1 = 5, n2 = 10, max;
    max = (n1 > n2) ? n1 : n2;
    cout << "Largest Number between"
    << n1 << " and " << n2 << " is " << max << endl;
    return 0;
}

```

Output :-

(Largest Number between 5 and 10 is = 10)

Use 11, 6

(Largest Number between 11 and 6 is = 11)

Use 1254, 3333

(Largest Number between 1254 and 3333 is = 3333)

11. C++ Arithmetic Operators:-

- Binary Operators
- Expression

Two Operands:-

- Operand 1
- Operand 2

Operand 1 = 1

Operand 2 = 2

Operator	Name	Example
+	Addition	1+2
-	Subtraction	1-2
*	Multiplication	1*2
/	Division	1/2
%	Modulus	1%2
++	Increment	++ 1
--	Decrement	-- 1

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
{
```

```
    int Var1 = 6;
```

```
    int Var2 = 12;
```

```
    int temp;
```

```
    temp = Var1 + Var2;
```

```
    cout << "Addition Result:" << temp << endl;
```

```
    temp = Var1 - Var2;
```

```
    cout << "Subtraction Result:" << temp << endl;
```

```
    temp = Var1 * Var2;
```

```
    cout << "Multiplication result:" << temp << endl;
```

```
    temp = Var1 / Var2;
```

```
    cout << "Division Result:" << temp << endl;
```

```
    temp = Var1 % Var2;
```

```
    cout << "Modulus result:" << temp << endl;
```

```
    temp = ++Var1;
```

```
    cout << "Increment Result:" << temp << endl;
```

```
    temp = --Var1;
```


cout << "Decrement Result;" << temp << endl;

return 0;

}

Output:

Addition Result: 18

Subtraction Result: -6

Multiplication Result: 72

Division Result: 0

Mod Result: 6

Increment Result: 6

Decrement Result: 7

12. C++ Unary Operators:-

Unary

One value (OSB operand 2SB operation perform krin g).

Binary

Two value (2SB operand 2SB operation perform krin g).

Unary Operator:-

Unary Operator are operators that act upon a single operand to produce a new value.)

Unary ~~minus~~ minus (-)

Increment (++)

decrement (--)

NOT (!)

Address of operator (&)

Size of ()

Unary minus:-

The minus operator changes the sign of its arguments. A positive number becomes negative, and a negative number becomes positive.

```
int a = 10;  
int b = -a; // b = -10
```

Unary minus is different from subtraction operator, as subtraction requires two operands.

Increment:

It is used to increment the value of the variable by 1. The increment can be done in two ways.)

Prefix increment:

In this method, the operator precedes the operand (e.g., ++a);

The value of operand will be altered before it is used.)

```
int a = 1;
```

```
int b = ++a; // b = 2
```

Postfix increment:

In this method, the operator follows the operand (e.g., a++). The value of operand will be altered after it is used.)

```
int a = 1;
```

```
int b = a++; // b = 1
```

```
int c = a; // c = 2
```

Decrement:

It is used to decrement the value of the variable by 1. The decrement can be done in two ways.)

Prefix decrement:

In this method, the operator precedes the operand (e.g., --a). The value of

Operand will be altered before it is used.

```
int a = 1;
```

```
int b = --a; // b = 0
```

Postfix decrement:

In this method, the operator follows the operand (eg., $a--$). The value of operand will be altered after it is used.)

```
(int a = 1;
```

```
int b = a--; // b = 1
```

```
int c = a; // c = 0)
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int a = 1;
```

```
cout << "a value: " << a << endl;
```

```
int b = a++;
```

```
cout << "b value after a++:" << b << endl;
```

```
cout << "a value after a++:" << a << endl;
```

```
// Pre increment
```

```
a = 1;
```

```
cout << "a value: " << a << endl;
```

```
b = ++a;
```

```
cout << "b value after ++a:" << b << endl;
```

```
cout << "a value after ++a:" << a << endl;
```

```
// post decrement
```

```
a = 5;
```

```
cout << "a value before decrement:" << a  
      << endl;
```

```
b = a--;
```

```
cout << "b value after a--:" << b << endl;
```

```
cout << "a value after a--:" << a << endl;
```

// Pre decrement

a = 5;

cout << "a value: " << a << endl;

b = --a;

cout << "b value after --a" << b << endl;

cout << "a value after --a" << a << endl;

return 0;

}

Output:

a value: 5

b value after a++: 1

a value after a++: 2

a value: 1

b value after ++a: 2

a value after ++a: 2

a value before decrement: 5

b value after a--: 5

a value after a--: 4

a value: 5

b value after --a: 4

a value after --a: 4

(NOT(!)):-

It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.

If x is true, then !x is false

If x is false, then !x is true

Address of operator (&);:-

It gives an address of a variable. It is used to return the memory address of a variable. These addresses returned by the address - of operator are known as pointers because they "point" to the variable in memory.)

Address of operator (&);:-

& gives an address on variable in

int a;

int * ptr;

ptr = &a; // address of a is copied to the location ptr.)

Size of ();:-

This operator returns the size of its operand in bytes. The size of operator always precedes its operands. The operand is an expression, or it may be a cast.)

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
{ // double, int
```

```
float n = 0;
```

```
cout << "size of n:" << sizeof(n);
```

```
return 1;
```

```
}
```

Output:-

size of : 4 → int - 4 bytes output dhan

use double

size of : 8

13. C++ Assignment Operators :-

Assignment Operators :-

The assignment operator, $=$, is the operator used for assignment... The copy assignment operator, often just called the "assignment operator", is a special case of assignment operator where the source (right-hand side) and destination (left-hand side) are of the same class type.)

(Simple assignment)	$a = b$
(addition assignment)	$a += b$
(Subtraction assignment)	$a -= b$
(multiplication assignment)	$a *= b$
(division assignment)	$a /= b$
(modulo assignment)	$a \% = b$
(bitwise AND assignment)	$a \& = b$
(bitwise OR assignment)	$a \mid = b$
(bitwise XOR assignment)	$a \wedge = b$
(bitwise left shift assignment)	$a \ll = b$
(bitwise right shift assignment)	$a \gg = b$

- $C = A + B$ will assign the value of $A + B$ to C
- $C += A$ is equivalent to $C = C + A$
- $C -= A$ is equivalent to $C = C - A$
- $C *= A$ is equivalent to $C = C * A$
- $C /= A$ is equivalent to $C = C / A$
- $C \% = A$ is equivalent to $C = C \% A$

• $C \gg = 2$ is same as $C = C \gg 2$

• $C \ll = 2$ is same as $C = C \ll 2$

• $C \& = 2$ is same as $C = C \& 2$

• $C \wedge = 2$ is same as $C = C \wedge 2$

• $C | = 2$ is same as $C = C | 2$

```
#include <iostream>
```

```
using namespace std;
```

```
main()
```

```
{
```

```
    int a = 21;
```

```
    int c;
```

```
    C = a;
```

```
    cout << " = operator, value of c =: " << c << endl;
```

```
    C += a;
```

```
    cout << " += operator, value of c =: " << c << endl;
```

```
    C -= a;
```

```
    cout << " -= operator, value of c =: " << c << endl;
```

```
    C *= a;
```

```
    cout << " *= operator, value of c =: " << c << endl;
```

```
C +=
```

```
    C /= a;
```

```
    cout << " /= operator, value of c =: " << c << endl;
```

```
    C = 200;
```

```
    C %= a;
```

```
    cout << " %= operator, value of c =: " << c << endl;
```

```
    C <<= 2;
```

```
    cout << " <<= operator, value of c =: " << c << endl;
```

$c >> = 2;$

cout << ">> = Operator, Value of c = : " << c << endl;

$c \& = 2;$

cout << "& = Operator, Value of c = : " << c << endl;

$c \wedge = 2;$

cout << " ^ = Operator, Value of c = : " << c << endl;

$c | = 2;$

cout << " | = Operator, Value of c = : " << c << endl;

return 0;

3)

Output :-

= Operator, Value of c = : 21

+ = Operator, Value of c = : 42

- = Operator, Value of c = : 21

* = Operator, Value of c = : 441

/ = Operator, Value of c = : 21

% = Operator, Value of c = : 11

<< = Operator, Value of c = : 44

>> = Operator, Value of c = : 11

& = Operator, Value of c = : 2

^ = Operator, Value of c = : 8

| = Operator, Value of c = : 2

14. C++ Relational Operators:-

Relational Operators:-

(In C++ Programming, the values stored in two variables can be compared using following operators and relation between them can be determined.)

- $>$, Greater than)
- $>=$, Greater than or equal to)
- $=$, is equal to)
- $!=$, is not equal to)
- $<=$, Less than or equal to)
- $<$, Less than)

```
#include <iostream>
using namespace std;
int main() {
    int a=10, b=20; c=10;
    if (a>b)
        cout << "a is greater" << endl;
    if (a<b)
        cout << "a is smaller" << endl;
    if (a<=c)
        cout << "a is less than / equal to c" << endl;
    if (a>=c)
        cout << "a is greater than / equal to c" << endl;
    return 0;
}
```

Output:-

a is smaller
a is less than / equal to a
a is ~~less~~ greater than / equal to c.

```
#include <iostream>
using namespace std;
int main() {
```

```
    int num 1 = 30;
    int num 2 = 40;
    int num 3 = 40;
    if (num 1 != num 2)
        cout << "Num 1 is not equal to Num 2" << endl;
    if (num 2 == num 3)
        cout << "Num 2 is equal Num 3" << endl;
    return 0;
}
```

Output:-

Num 1 is not equal to Num 2
Num 2 is equal Num 3

15. C++ Logical Operators :-

Logical Operators :-

(Logical Operators are used to combine two or more conditions / constraints or to complement the evaluation of the original condition in consideration. The result of the operation of a logical operator is a boolean value either true or false.)

Operators	Name of the operator
&&	AND operator
	OR operator
!	NOT operator

Operator	1st condition	2nd condition	Output
AND	True	True	True
	True	False	False
	False	True	False
	False	False	False
OR	True	True	True
	True	False	True
	False	True	True
	False	False	False
NOT	True	-	False
	False	-	True

```

#include <iostream>
using namespace std;
int main()
{
    int num1 = 30;
    int num2 = 40;
    if (num1 >= 40 || num2 >= 40)
        cout << "OR if block gets executed" << endl;
    if (num1 >= 20 && num2 >= 20)
        cout << "AND if block gets executed" << endl;
    if (! (num1 >= 40))
        cout << "NOT if block gets executed" << endl;
    return 0;
}

```

Output:-

OR if block gets executed
AND if block gets executed
NOT if block gets executed

16. C++ If Statements:-

● In C++ Programming, if statement is used to test the condition. There are various types of if statements in C++.

● If statement

- if-else statement
- nested if statement
- If-else-if ladder

● C++ If statement:-

If statement tests the condition. It is executed if condition is true.)

```
if (condition) {
```

```
    // Code to be executed
```

```
}
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int x = 20;
```

```
    int y = 10;
```

```
    if (y > x)
```

```
    {
```

```
        cout << "y is greater than x";
```

```
    }
```

```
    return 0;
```

```
}
```

Output:-

y is greater than x.


```
#include <iostream>
using namespace std;
```

```
int main()
```

```
{
```

```
    int x = 30;
```

```
    int y = 30;
```

```
    if (y == x)
```

```
    {
```

```
        cout << "x and y are equal";
```

```
    }
```

```
    return 0;
```

```
}
```

Output:-

x and y are equal

17. C++ If else Statements:-

(C++ IF-else Statement:-

The c++ if-else statement also tests the condition. It executes if block if condition is true otherwise else block is executed.)

```
if (Condition) {
```

```
    // Code if condition is true
```

```
} else {
```

```
    // Code if Condition is false
```

```
}
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int num = 11;
```

```
    if (num % 2 == 0)
```

→ 10

```

{
    cout << "The Number is EVEN";
}
else
{
    cout << "The number is ODD";
}
return 0;
}

```

Output :-

The number is ODD

Use 10 (The output is)

The number is EVEN

```

#include <iostream>

```

```

using namespace std;

```

```

int main()

```

```

{

```

```

    int num;

```

```

    cout << "Enter a Number!";

```

```

    cin >> num;

```

```

    if (num % 2 == 0)

```

```

    {

```

```

        cout << "The Number is Even";

```

```

    }

```

```

    else

```

```

    {

```

```

        cout << "The number is ODD";

```

```

    }

```

```

    return 0;

```

```

}

```

Output :-

Enter the Number: 5

The number is ODD


```

#include <iostream>
using namespace std;
int main()
{
    int i = 10;
    if (i == 10)
    {
        if (i < 15)
        {
            cout << "number is smaller than 15\n";
            if (i < 12)
            {
                cout << "Number is smaller than 12\n";
            }
            else
            {
                cout << "number is not less than 12\n";
            }
        }
        return 0;
    }
}

```

Output:-

Number is smaller than 15
 number is smaller than 12

18. C++ If else if Ladder :-

(C++ If - else - If ladder Statement :-)

(The c++ if - else - if ladder statement executes one condition from multiple statements.)

```

if (condition 1) {
    // Code to be executed if condition 1 is true
} else if (condition 2) {
    // Code to be executed if condition 2 is true
} else if (condition 3) {
    // Code to be executed if condition 3 is true
}

```

else {

// Code to be executed ^{cu} if all the conditions
are false

}

#include <iostream>

using namespace std;

int main() {

int num;

cout << "Enter a number to check grade";

cin >> num;

if (num < 0 || num > 100)

{

cout << "wrong number";

}

else if (num >= 0 && num < 50)

{

cout << "fail";

}

else if (num >= 50 && num < 60)

{

cout << "D grade";

}

else if (num >= 60 && num < 70)

{

cout << "C grade";

}

else if (num >= 70 && num < 80)

{

cout << "B grade";

}

else if (num >= 80 && num < 100)

{

cout << "A grade";

}

}

```
else if (num >= 90 && num <= 100)
```

```
{
```

```
    cout << "A+ grade";
```

```
}
```

```
    return 0;
```

```
}
```

Output:-

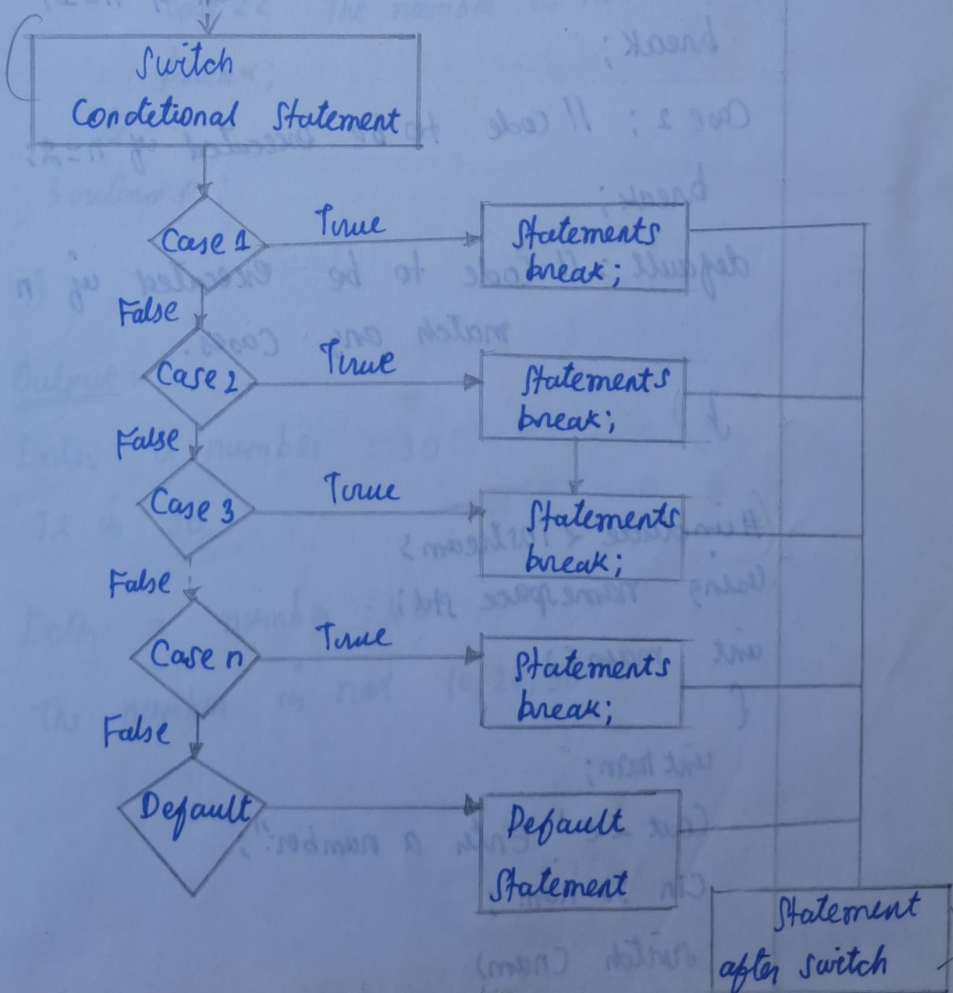
Enter a number to check grade 40

Fail

19. C++ Switch Statement:

(C++ Switch:-

The C++ switch statement executes one statement from multiple conditions. It is like if-else-if ladder statement in C++.)




```
Switch (expression) {
```

```
Case value 1:
```

```
// Code to be executed;
```

```
break;
```

```
Case value 2:
```

```
// Code to be executed
```

```
break;
```

```
default:
```

```
// Code to be executed if all cases are not  
matched;
```

```
break;
```

```
}
```

```
(Switch (n)
```

```
{
```

```
Case 1: // Code to be executed if  $n=1$ ;  
break;
```

```
Case 2: // Code to be executed if  $n=2$ ;  
break;
```

```
default: // Code to be executed if  $n$  doesn't  
match any cases.
```

```
} )
```

```
(#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int num;
```

```
cout << "Enter a number:";
```

```
cin >> num;
```

```
switch (num)
```

{

Case 10:

{

cout << "it is 10";

break;

}

Case 20:

{

cout << "It is 20";

break;

}

Case 30:

{

cout << "it is 30";

break;

}

default:

{

cout << "The number is not 10, 20, 30";

break;

}

return 0;

3)

Output:-

Enter a number : 30

It is 30

Enter a number : 40

The number is not 10, 20, 30

• braces illama
Kooda type
Pannalam
(Join one line)
- ~~Print~~
- ~~Print~~
(Program
backside)

```

#include <iostream>
using namespace std;
int main()
{
    int num;
    cout << "Enter a number:";
    cin >> num;
    switch (num)
    {
        case 10: cout << "It is 10"; break;
        case 20: cout << "It is 20"; break;
        case 30: cout << "It is 30"; break;
        default: cout << "The number is not 10,20,30";
                break;
    }
    return 0;
}

```

Output:-

Enter a number: 30
It is 30)

20. C++ Function Tutorial:-

(C++ Functions:-

- Set of statements
- main())

(Function Definition:-

- Type
- Name
- Parameters
- Body)

Function definition:-

```
( type name (parameter)
{
    body
}
```

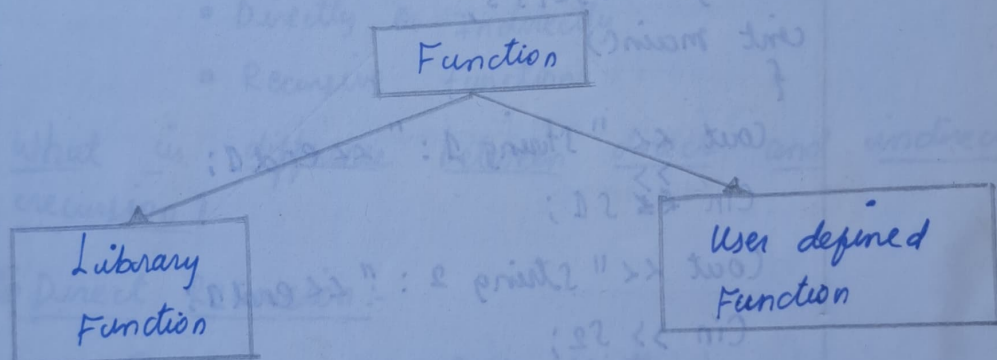
Function Declarations:-

```
( • Type
  • Name
  • parameter )
```

Function declaration Syntax:-

```
( type name (parameter);
```

Function Types:-



```
(#include <iostream>
```

```
using namespace std;
```

```
void function 1();
```

```
void function 2();
```

```
int main()
```

```
{
```

```
    function 1();
```

```
    function 2();
```

```
    return 0;
```

```
}
```

```
void function ()
```

```
{
```

```

    cout << "Programming Line";
}

void function2()
{
    cout << "Line";
}

```

Output:

Programming Line

```

#include <iostream>
#include <string.h>
using namespace std;

```

```

void function1 (string, string);
string s1, s2, s3;
int main()
{

```

```

    cout << "String 1: " << endl;
    cin >> s1;

```

```

    cout << "String 2: " << endl;
    cin >> s2;

```

```

    function1 (s1, s2);
    return 0;
}

```

```

void function1 (string a, string b)
{

```

```

    s3 = s1 + s2;

```

```

    cout << s3 << endl;

```

```

}

```

String: " << s3 << endl;

Output:

String 1:

Programming

String 2:

Line

Programming Line

Output:-

String 1 : Programming

String 2 : Line

Concatenated String : Programming Line

2). C++ Function Recursion :-

Recursion in C++ :-

(What is Recursion?)

- Function calls itself
- Directly or Indirectly
- Recursive function

What is difference between direct and indirect recursion?

● Direct Recursion:-

Function calls itself directly

● Indirect Recursion:-

Function calls a new function and that function calls back the old function.

Direct Recursion

(Void Function C)

{

Function C;

})

Indirect Recursion

void FunctionOld()

{

FunctionNew();

}

void FunctionNew()

{

FunctionOld();

}

```
#include <iostream>
using namespace std;
```

int main()

{

int num;

cout << "Number: ";

cin >> num;

cout << "Factorial: " <<

return 0;

}

```
#include <iostream>
```

```
using namespace std;
```

int function1 (int Number)

{

if (Number <= 1)

return 1;

else

return Number * function1 (Number - 1);

}

int main()

{

int num;

cout << "Number: ";

```
Cin >> num;
```

```
cout << " Factorial : " << function1(num);
```

```
return 0;
```

3
Output:-

Number : 3

• factorial concept = $3 = 1 * 2 * 3$

$4 = 1 * 2 * 3 * 4$

$5 = 1 * 2 * 3 * 4 * 5$

```
#include <iostream>
```

```
using namespace std;
```

```
int functionOld(int);
```

```
int functionNew(int);
```

```
int functionOld (int number)
```

```
{  
    if (number <= 1)
```

```
        return 1;
```

```
    else
```

```
        return number * functionNew (number - 1);
```

```
}
```

```
int functionNew (int Number)
```

```
{  
    if (Number <= 1)
```

```
        return 1;
```

```
    else else
```

```
        return number * functionOld (number - 1);
```

```
}
```

```
int main()
```

```
{
```

```
    int num = 5;
```

```
    cout << functionOld (num);
```

```
    return 0;
```

```
}
```

Can use

[cout << "Factorial : "

<< functionOld
(num);]

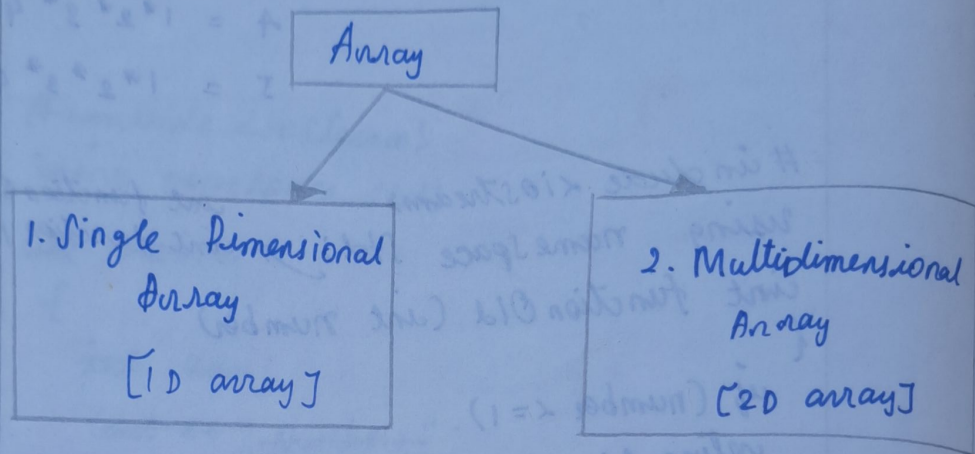
Output:-

120

22. C++ Arrays:-

What is an array in C++?

- Collection of items
- Contiguous memory locations
- Indices
- Primitive data



Array declaration Syntax:-

- Type name [elements];

Array declaration:-

int sample[8];

Initializing arrays:-

int sample[8] = {11, 12, 13, 14, 15, 16, 17, 18};

11	12	13	14	15	16	17	18
0	1	2	3	4	5	6	7

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main()
```



```

{
    String sample[5] = {"Programming", "line", "youtube",
                        "google", "facebook"};
    for (int i=0; i<5; i++)
    {
        cout << "array [" << i << "] - " << sample[i]
              << "\n";
    }
    return 0;
}

```

Output:-

array[0] - Programming
 array[1] - line
 array[2] - youtube
 array[3] - google
 array[4] - facebook)

Multi dimensional array:

```
int sample[3][3];
```

0	1	2	
1	2	3	0
4	5	6	1
7	8	9	2

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int array[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}};
```

```
for (int i=0; i<3; i++)
```

```
for (int j=0 ; j<3 ; j++)
```

```
{
```

```
    cout << "array [" << i << "]" << j << "]" ;
```

```
    cout << array [i][j] << endl ;
```

```
}
```

```
return 0 ;
```

```
}
```

Output:-

array [0][0] 1

array [0][1] 2

array [0][2] 3

array [1][0] 4

array [1][1] 5

array [1][2] 6

array [2][0] 7

array [2][1] 8

array [2][2] 9

23.

C++ pointers:-

What is a pointer in C++?

- Variable
- Points to address
- Locator or indicator

Symbols used in pointer?

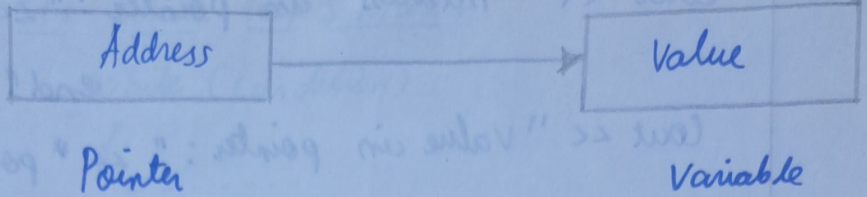
& (ampersand sign)

* (asterisk sign)

How to declare a pointer?

int * pointer, Var;

pointer = &Var;



```
#include <iostream>
using namespace std;

int main()
{
    int * pointer;
    int num = 50;
    pointer = &num;
    cout << "Variable address: " << &num << endl;
    cout << "Address stored in pointer: " << pointer
    << endl;
    cout << "Value stored in variable: " << *pointer
    << endl;
    return 0;
}
```

Output:-

Variable address: 0x7ffe637767c4
Address stored in pointer: 0x7ffe637767c4
Value stored in variable: 50

```
#include <iostream>
using namespace std;

int main()
{
    int var, * pointer;
    var = 5;
    cout << "Variable Address: " << &var << endl;
    cout << "Variable Value: " << var << endl;

    pointer = &var;
}
```



```
cout << "Address in pointer:" << pointer << endl;
```

```
cout << "Value in pointer:" << *pointer << endl;
```

```
Var = 55;
```

```
cout << "Address in pointer:" << pointer << endl;
```

```
cout << "Value in pointer:" << *pointer << endl;
```

```
*Pointer = 10;
```

```
cout << "Variable Address:" << &Var << endl;
```

```
cout << "Variable Value:" << Var << endl;
```

```
return 0;
```

```
}
```

Output:-

Variable Address : 0x7ffdfc3cd004

Variable Value : 5

Address in pointer : 0x7ffdfc3cd004

Value in pointer : 5

Address in pointer : 0x7ffdfc3cd004

Value in pointer : 55

Variable Address : 0x7ffdfc3cd884

Variable Value : 10

24. C++ while and do while loop:-

(C++ while loop:-

• loop - iteration

• Iteration

• Iteration is not fixed

• Condition ^{is} true)

C++ While loop Syntax :-

while (condition)

{

}

C++ do while loop :-

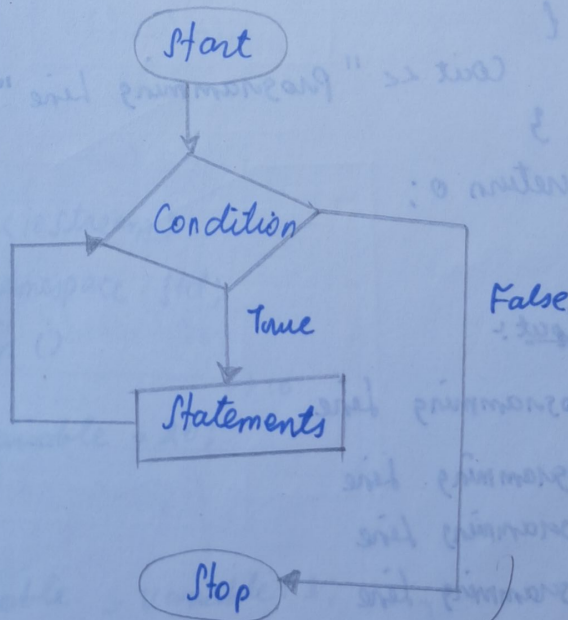
- Iteration
- Iteration is not fixed
- Condition is true
- Execute the loop at least once.

C++ do while loop Syntax :-

do

{

} while (condition);




```

#include <iostream>
using namespace std;

int main()
{
    int var = 1;
    while (var <= 10)
    {
        cout << var << endl;
        var++;
    }
    return 0;
}

```

Output:-

1
2
3
4
5
6
7
8
9
10

```

#include <iostream>
using namespace std;

int main()
{
    int variable = 20;
    do
    {
        variable = variable + 1;
        cout << variable << endl;
    } while (variable <= 10);

    return 0;
}

```

By Bunce 20 dhan
Print 20 to 30.

Output:-

21

25.

(C++ for loop:-

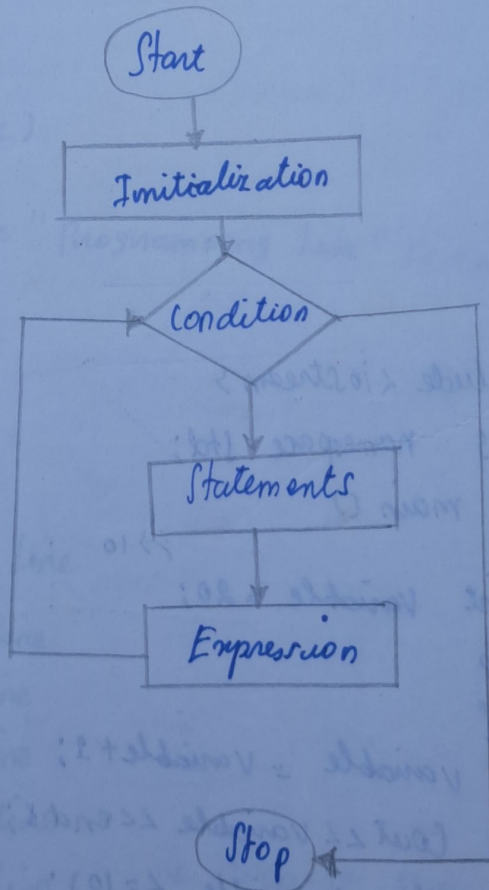
(C++ For loop:-

- Loop that is executed a specific number of times
- efficient loop.)

(for loop syntax:-

for (initialization ; condition ; increment)
{

}



(#include <iostreams

using namespace std;

int main()

`{ >> num >> " " → 1000, 1 " → 100, 15 → i-100, i++`

`for (int i=100; i<=1000; i+=100)`

`{ cout << i << " \n"; } "Programming line"`

`}`

`}`

Output :-

100	1000	Programming line	} 15 times Print 2600.
200	900	Programming line	
300	800	Programming line	
400	700	Programming line	
500	600	Programming line	
600	500	Programming line	
700	400	Programming line	
800	300	Programming line	
900	200	Programming line	
1000	100	Programming line	

26. C++ break statement :-

(C++ break statement :-

- Terminate
- Nested loop

`#include <iostream>`

`using namespace std;`

`int main()`

`{`

`for (int num = 100; num <= 1000; num += 100)`

`{`

`if (num == 500)`

`{`

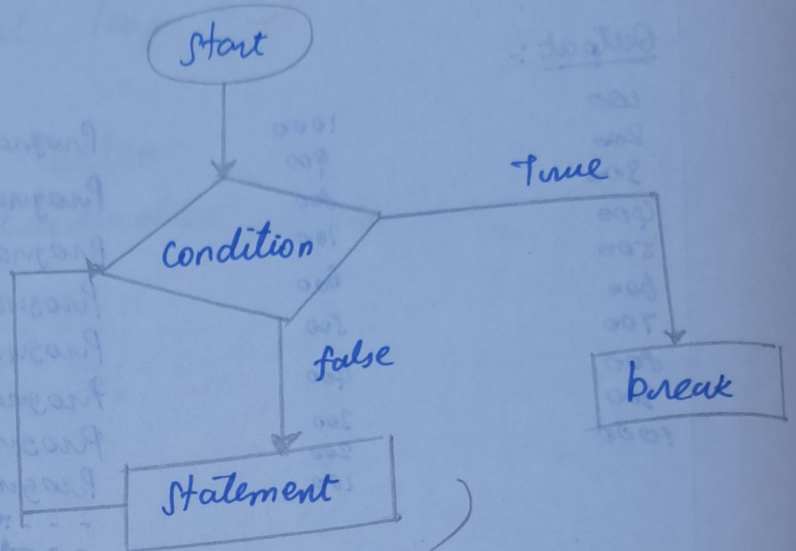
`break;`

`}`


```

    cout << "Number: " << num << "\n";
    }
    return 0;
}

```



```

for (int i=1; i<=10; i++)
{
    if (i==5)
    {
        break;
    }
}

```

```

    cout << i << endl;
}

```

```

#include <iostream>
using namespace std;
int main()
{
    int num = 100;
    do
    {
        if (num > 500)

```

```

{
    break;
}

cout << " Number : " << num << endl;
num = num + 100;
} while (num < 1000);
return 0;
}

```

Output:-

```

Number : 100
Number : 200
Number : 300
Number : 400
Number : 500

```

27. C++ String:-

C++ Strings:-

- Sequence of characters
- header file - <string>

C++ Syntax:-

```

#include <string>

string Variable = "Programming Line";

#include <iostream>
#include <string>

using namespace std;

int main()
{
    String var 1 = "Programming";
    String var 2 = "Line";
    String temp = var 1 + " " + var 2;
    cout << temp; >> " Result"
    return 0;
}

```


3

Output :

Programming Line

use result

Result : Programming Line

(String Length:-

- length()
- size()

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    string Var = "Programming Line";
```

```
    cout << "Length of the string:" << Var.
```

size(); → length();

```
    return 0;
```

```
}
```

Output :-

Length of the string is : 16

can use length():

Length of the string: 16

Programming

Length of the string : 11

28.

C++ Class & Objects :-

Class

- user-defined data type
- Class (data members, member functions)
- accessed by instance

Object

- Instance of a class

Class definition

```
class class_name
```

```
{
```

```
Access specifier:
```

```
Data member;
```

```
Member function ()
```

```
{
```

```
-----
```

```
}
```

```
}
```

Object declaration:-

```
class_name object_name;
```

```
#include <iostream>
```

```
using namespace std;
```

```
class sample
```

```
{
```

```
public;
```

```
String name;
```

```
int roll;
```

```
int age;
```

```
String dept;
```

```
void display name();
```

```

Void displayid()
{
    Cout << "Id : " << roll << endl;
}

Void display age()
{
    Cout << "Age : " << age << endl;
}

Void displaydept()
{
    Cout << "Department : " << dept << endl;
}

};

Void Sample :: displayname()
{
    Cout << "Name : " << name << endl;
}

int main()
{
    Sample obj1;
    Obj . name = "vishal";
    Obj 1 . roll = 28;
    Obj 1 . age = 21;
    Obj 1 . dept = "Information Technology";
    Obj 1 . displayname();
    Obj 1 . displayid();
    Obj 1 . displayage();
    Obj 1 . displaydept();
    return 0;
}

```

Output :

Name : Vishal

Id : 28

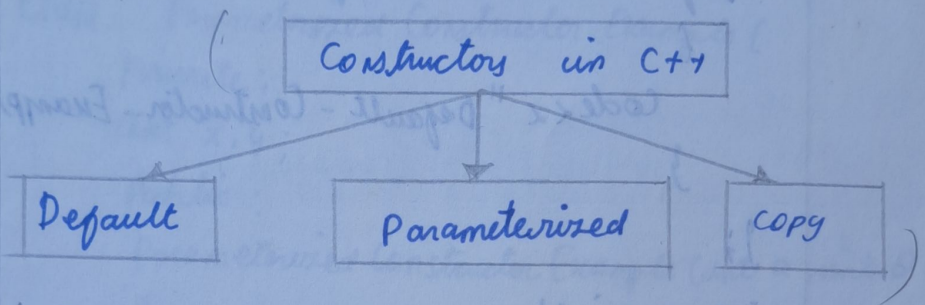
Age : 21

Department : Information Technology

29. C++ Constructor :-

(C++ Constructor:-

- Constructor is a Function
- Invoked Automatically
- Use the same name as the class
- No return type.



#include <iostream>

using namespace std;

Class Default Constructor Example {

public:

int x, y;

Default Constructor Example()

{

 x = 3;

 y = 6;

}

};

int main()

{

 Default Constructor Example obj;

 cout << "Value of x:" << obj.x << endl;

 cout << "Value of y:" << obj.y;

 return 0;

}
Output:-

Value of x : 3

Value of y : 6

```
#include <iostream>
```

```
using namespace std;
```

```
Class Default Constructor Example{
```

```
Public;
```

```
Default Constructor Example{
```

```
{
```

```
Code << "Default - Constructor - Example";
```

```
}
```

```
};
```

```
int main{
```

```
{
```

```
Default Constructor Example obj;
```

```
return 0;
```

```
}
```

Output:-

Default constructor Example

```
#include <iostream>
```

```
using namespace std;
```

```
Class Default Constructor Example{
```

```
Public :
```

```
Void func()
```

```
{
```

```
Code << "Normal - function";
```

```
}
```

```
};
```

```
int main()
```

```
{  
    // default constructor Example : Obj;  
    Obj.fun();  
    return 0;  
}
```

Output:-

Normal Function)

```
#include <iostream>
```

```
using namespace std;
```

```
Class Parameterized Constructor Example {
```

```
Private;
```

```
int x, y;
```

```
Public:
```

```
Parameterized Constructor Example (int a, int b)
```

```
{
```

```
    x = a;
```

```
    y = b;
```

```
}
```

```
int x() {
```

```
{
```

```
    return x;
```

```
}
```

```
int y() {
```

```
{
```

```
    return y;
```

```
}
```

```
};
```

```
int main
```

```
{
```

```
    Parameterized Constructor Example obj(100, 200);
```

```
    cout << "value of x: " << obj.x() << endl;
```

```
    cout << "value of y: " << obj.y();
```

```
    return 0;
```

```
}
```


Output:-

Value of x: 100
Value of y: 200

```
#include <iostream>
using namespace std;
```

```
Class CopyConstructor Example {
public;
```

```
int x;
```

```
Copy Constructor Example (int a)
{
```

```
    x = a;
```

```
}
```

```
CopyConstructor Example (CopyConstructor Example  
&b)
```

```
{
```

```
    x = b.x;
```

```
}
```

```
};
```

```
int main ()
```

```
{
```

```
    CopyConstructor Example Obj1 (200);
```

```
    CopyConstructor Example Obj2 (Obj1);
```

```
    cout << Obj2.x;
```

```
    return 0;
```

```
}
```

Output:-

200

30. C++ Destructor:-

- Destructor is a function
- Destroys or Deletes an object
- Called automatically

```

#include <iostream>
using namespace std;

class DestructorExample {
    private;
    int a, b;
    public;
    DestructorExample(int x, int y) {
        cout << "1" Constructor \\" << endl;
        a = x;
        b = y;
    }

    void output() {
        cout << "A = " << a << endl;
        cout << "B = " << b << endl;
    }

    ~DestructorExample() {
        cout << "1" Destructor \\";
    }
};

int main() {
    DestructorExample obj(100, 200);
    obj.output();
    return 0;
}

```

Output:-

" Constructor "

A = 100

B = 200

" destructor "

31.

C++ Scope Resolution Operator:-

- Scope resolution operator (::)
- global variable.
- define a function outside a class.)

```
#include <iostream>
#include <string>
using namespace std;
String var1 = "Programming";
Static int var2 = 10;
int main()
{
    String var1 = "Line";
    Cout << "Value of Global Integer Variable : "
        << ::var2 << endl;
    Cout << "Value of Global String Variable : "
        << ::var1 << endl;
    Cout << "Value of Stored in Local Variable : "
        << var1 << endl;
    return 0;
}
```

Output:-

Value of Global Integer variable: 10
 Value of Global String Variable: Programming
 Value stored in Local Variable: Line)

```
#include <string>
using namespace std;
Class Sample
```

```

{
    public:
        String name;
        int id;

        Void first();
        Void second();
    {
        cout << "Employee Id : " << id;
    }
};

void Sample :: first()
{
    cout << "Employee Name : " << name << endl;
}

int main()
{
    Sample obj1;
    obj1.name = "Vishal";
    obj1.id = 10;
    obj1.first();
    obj1.second();
    return 0;
}

```

Output :-

Employee Name : Vishal

Employee Id : 10